

PSR-PIPELINE: A Reproducible sEMG Benchmarking Workflow for Finger Intent Prediction in Post-Stroke Rehabilitation

Repository-derived manuscript draft

July 11, 2026

Abstract

Reliable finger-intent decoding from surface electromyography (sEMG) is attractive for post-stroke rehabilitation, but it is difficult to study reproducibly when preprocessing choices, tensor layouts, and model evaluation code are scattered across separate scripts. This paper documents PSR-PIPELINE, a repository-centered workflow that packages data harmonization, a three-stage sEMG preprocessing stack, a twelve-feature window representation, shared model adapters, and benchmark utilities for multi-label finger-intent prediction on PHYSIOMIO [7, 1]. The current benchmark path aligns gesture labels with raw EMG, filters the signal with a 20–450 Hz zero-phase Butterworth stage, denoises it with wavelet soft-thresholding, segments 200 ms windows with 50% overlap, and converts each window into a channel-by-feature tensor that can be consumed by LSTM, CNN, and graph neural network predictors. On the repository’s held-out test artifacts, the LSTM achieves the strongest exact-match accuracy, finger accuracy, and macro AUROC (0.545, 0.784, and 0.858), while the GNN achieves the strongest macro F1 and AUPRC (0.706 and 0.776). The updated manuscript also records two open repository extensions: PR #56 modularizes the 1D CNN training stack and latency-aware optimization, while PR #59 adds teacher-ensemble distillation, threshold tuning, and local CPU latency benchmarking; because both are still under review, they are treated as pending extension evidence rather than merged headline results [5, 6]. These results position PSR-PIPELINE as a useful systems benchmark rather than a single-model state-of-the-art claim, because the available evidence still covers one dataset and does not yet include full ablation or cross-dataset generalization studies. Even with that limitation, the released pipeline, metrics, pull-request notes, and paper-support artifacts establish a concrete base for future rehabilitation-oriented sEMG experiments.

1 Introduction

Finger-intent prediction from sEMG is a clinically meaningful interface problem because it can connect residual forearm muscle activity to assistive control, therapy feedback, and staged post-stroke motor retraining [10]. In practical terms, the task is to recover a five-finger activation pattern from multichannel forearm sEMG so that downstream rehabilitation software or hardware can react to a user’s intended movement.

The technical difficulty is not only model accuracy, but also pipeline reproducibility. Post-stroke sEMG is noisy, weakly structured, and sensitive to subject-specific recording conditions, so reported performance depends heavily on filtering, denoising, segmentation, label construction, and data splitting. Public repositories often expose only part of that path, which makes it hard to compare alternative predictors under a consistent signal-processing and evaluation protocol.

PSR-PIPELINE addresses that gap by organizing the full workflow in one codebase [7]. The repository

combines gesture-label harmonization, preprocessing modules, feature extraction, tensor adapters, dataset loaders, three predictor families, and saved benchmark metrics. It also includes additional two-stage healthy-to-impaired training scripts and summary files, while the two currently open pull requests extend the CNN branch with a cleaner 1D training stack, teacher-student distillation, threshold tuning, and latency benchmarking [5, 6]. Those extensions are relevant to the system story, but they are still treated separately from the merged benchmark evidence in this manuscript.

The central story of this paper is therefore systems-oriented rather than novelty-driven. We do not claim a new decoding architecture. Instead, we document a reusable benchmark workflow that makes the following engineering choices explicit: a fixed preprocessing specification, a shared window-level representation for model comparison, patient-level data splitting to reduce leakage, and repository-local metric artifacts that can be audited after training.

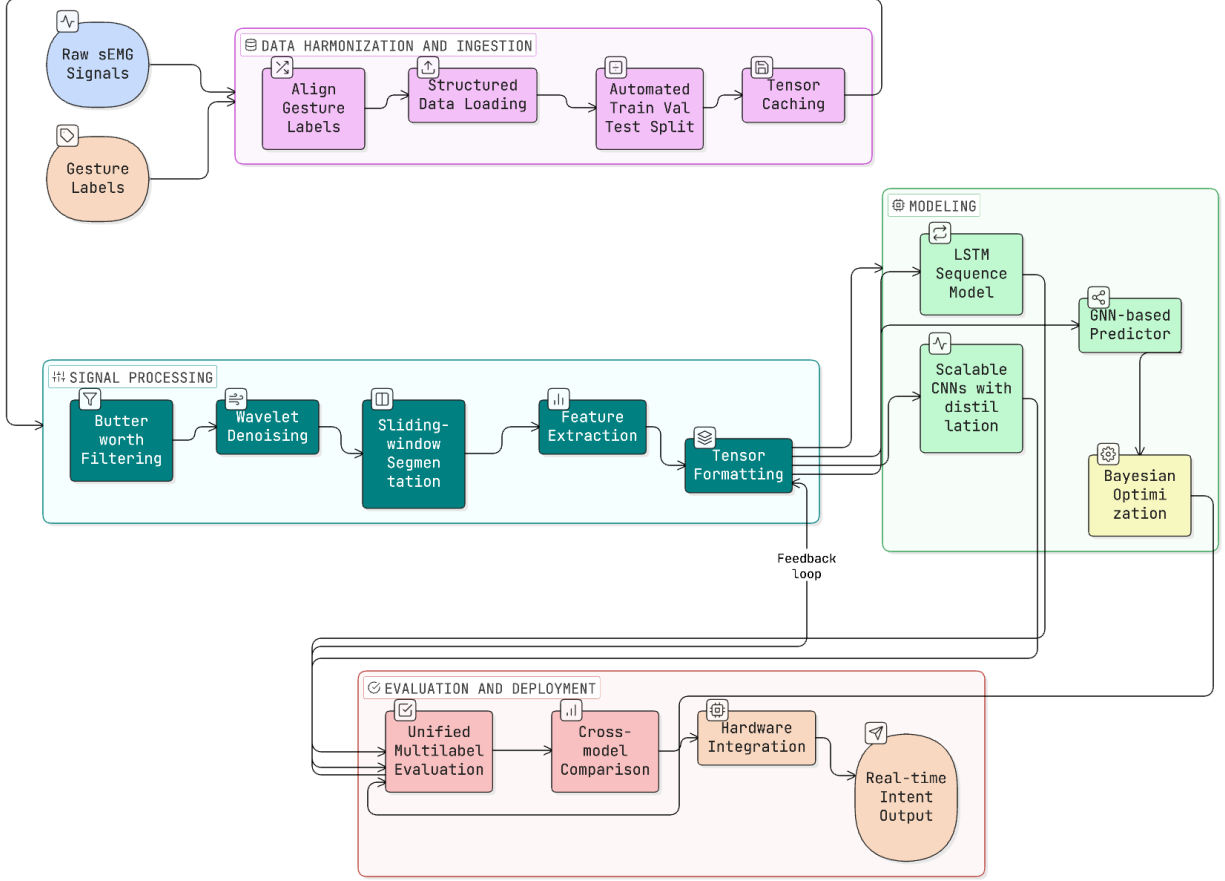
This updated paper makes five contributions. First, it formalizes the current preprocessing and feature pipeline implemented in the repository. Second, it explains how the code standardizes inputs so that LSTM, CNN, and GNN models can be compared inside one training and evaluation framework. Third, it consolidates saved metrics from all three predictor families, including the repository’s existing LSTM artifacts that are not highlighted in the top-level README table. Fourth, it records the two open CNN-focused pull requests as under-review system extensions, covering modular 1D training, distillation, threshold tuning, and latency benchmarking without overstating them as merged benchmark evidence. Fifth, it reports the present scope limits of the benchmark so that future work can extend the repository with clearer ablations, stronger transfer studies, and broader rehabilitation validation.

2 Related Work

sEMG has long been studied as a non-invasive control channel for rehabilitation, prosthetics, and human-machine interaction, with performance shaped by both acquisition quality and downstream decoding choices [10]. That literature makes clear that signal conditioning is not a minor implementation detail: denoising, spectral shaping, and feature design directly affect the reliability of any intent-decoding model.

Sequence-oriented deep models remain a natural fit for windowed biosignal decoding because they can aggregate temporal evidence across successive frames. In particular, LSTM-based models are well suited to sequential dependencies in noisy measurements [3], while compact 1D CNNs can emphasize local temporal motifs and efficient inference. The current repository follows both lines by exposing a recurrent baseline and several CNN size variants under one interface.

Graph neural networks provide a complementary viewpoint by modeling structured relationships among windowed observations rather than treating the sequence only as a flat timeline [4, 2, 8]. PSR-PIPELINE uses that idea to construct a graph over window-level nodes and then applies configurable graph operators before graph-level pooling. Relative to these prior model families, the distinguishing contribution here is not a new graph formulation or recurrent block, but a repository-level benchmark that keeps preprocessing, tensorization, and metric reporting consistent across heterogeneous predictors.



 eraser

Figure 1: Updated paper-level system diagram. The figure preserves the shared benchmark path used in this manuscript and surfaces adjacent engineering stages discussed in the repository and open PRs, including label alignment, tensor caching, CNN distillation support, and deployment-oriented evaluation hooks [5, 6].

3 Method

3.1 Overview

PSR-PIPELINE implements a modular decoding workflow that starts from raw multichannel sEMG and gesture labels and ends with either five sigmoid-compatible logits or downstream evaluation artifacts. The method is easiest to read as four connected bands: data harmonization and ingestion, signal processing and feature extraction, model fitting, and evaluation or deployment-facing analysis. Figure 1 summarizes the repository pipeline and the open-extension details incorporated into this updated paper revision.

3.2 Data harmonization and ingestion

Before filtering begins, the repository aligns gesture labels with raw PhysioMio recordings, structures subject-level examples, performs deterministic patient-level splitting, and caches processed tensors for reuse. These ingest operations matter because they define the effective task boundary: which arm is selected, how segment labels are mapped into five binary finger targets, and whether train-test leakage is avoided by splitting patients rather than windows. The updated paper figure makes these steps explicit because they are distributed across loaders and processing utilities rather than hidden inside one monolithic script.

3.3 Signal preprocessing

The preprocessing module is explicitly parameterized by a frozen configuration object in the repository. Raw sEMG first passes through a fourth-order Butterworth band-pass filter with 20–450 Hz cutoffs and zero-phase forward-backward application. This stage suppresses low-frequency motion artifacts and high-frequency electrical noise while preserving time alignment across channels.

The filtered signal is then denoised with wavelet soft-thresholding. The default configuration uses a Symlet-4 basis, four decomposition levels, universal threshold selection, and a median-based noise estimate. Although the repository does not yet include an ablation isolating this choice, the implementation makes the denoising rule explicit and reproducible.

After denoising, the continuous recording is segmented into 200 ms windows with 50% overlap. These window settings balance temporal responsiveness with a sufficient sample count for stable summary statistics. The repository disables zero-padding by default, so each window represents observed signal rather than synthetic boundary content.

3.4 Window-level feature representation

Each channel-window pair is summarized by twelve handcrafted descriptors spanning time and frequency domains. The time-domain set includes RMS, mean absolute value, integrated EMG, waveform length, variance, zero crossings, slope sign changes, and Willison amplitude. The frequency-domain set includes mean frequency, median frequency, spectral entropy, and total power, with spectral quantities estimated from Welch periodograms [9]. The resulting tensor has shape (C, W, F) , where C is the number of channels, W is the number of windows, and $F = 12$ is the feature count.

3.5 Tensor formatting and shared adapter

The repository standardizes cross-model comparison with one shared adapter. It transforms (C, W, F) tensors into $(N, W, C \times F)$ sequences, where windows act as time steps and all channel features are concatenated into one vector per step. This small interface decision matters because it allows recurrent, convolutional, and graph-based predictors to consume the same derived representation without duplicating preprocessing code. For the CNN branch, the baseline path then permutes the tensor into a channels-first layout, while open PR #56 makes that step more explicit by flattening the 64×12 channel-feature grid into 768 one-dimensional input channels for modular student and teacher training [5].

3.6 Predictor families

The LSTM branch applies two stacked recurrent layers with hidden sizes 128 and 256, followed by a 128-unit fully connected layer and dropout before five output logits. This branch emphasizes temporal accumulation across windows and, in the saved metrics, yields the strongest exact-match accuracy, finger accuracy, and AUROC.

The CNN branch wraps several 1D convolutional student models behind a channels-first permutation layer. In the current training code, the benchmarked CNN consumes the shared window-level sequence after permutation from $(N, W, C \times F)$ to $(N, C \times F, W)$. The repository supports multiple model sizes, but the saved benchmark artifacts are presented as one CNN baseline rather than a full architecture sweep. Open PR #56 restructures this branch into separate student, teacher, training, distillation, and optimization modules, and open PR #59 extends it with teacher ensembles plus compact Nano and Micro student variants aimed at edge deployment [5, 6].

The GNN branch interprets each time window as a node in a graph. The implementation supports GCN, GraphSAGE, and GAT operators [4, 2, 8]; the published repository narrative highlights a two-layer, 64-hidden-dimension configuration with mean pooling. In the current code, graphs are built densely across windows within each sample, enabling message passing over the full temporal context before graph-level pooling and readout.

3.7 Training, tuning, and evaluation protocol

The training utilities unify optimization and reporting across model families. The repository uses binary cross-entropy with logits for five-label prediction, Adam-based optimization, plateau-triggered learning-rate reduction, checkpointing, and saved metric curves. Evaluation reports exact-match accuracy, per-finger averaged accuracy, macro precision, macro recall, macro F1, macro AUROC, and macro AUPRC. The metrics module also stores per-finger scores and task-level curve data, which makes both aggregate and finger-wise analysis available after training.

The paper figure includes a Bayesian-optimization block because tuning is already a real part of the repository workflow rather than a paper embellishment. Current mainline code exposes Optuna-based search utilities for GNN and two-stage training paths, and the open CNN refactor in PR #56 turns that tuning layer into a latency-aware optimization stage for student size and training settings [5]. PR #59 further adds validation-driven threshold tuning and local CPU latency benchmarking, which are relevant for deployment screening even though they are not yet integrated into the main benchmark tables [6].

4 Experiments

4.1 Dataset and split construction

All benchmark artifacts analyzed in this paper come from the repository’s PHYSIOMIO processing path [1, 7]. The data loader searches raw parquet files, extracts contiguous movement segments, maps gesture labels to five-bit finger targets, preprocesses each segment, and stores processed tensors as PyTorch split files.

The split logic is patient-level rather than segment-level. Patients are shuffled with a fixed seed and divided into 70% train, 10% validation, and 20% test partitions, which reduces identity leakage between train and test segments. The default loader configuration targets impaired-arm recordings

unless the user explicitly changes the arm-split setting, so the main benchmark should be interpreted as a post-stroke impaired-arm decoding study.

4.2 Implementation scope

The repository exposes a richer experimental surface than the evidence-complete benchmark summarized in this paper. In addition to direct LSTM, CNN, and GNN runs, the codebase contains healthy-to-impaired pretraining and finetuning scripts, optional threshold tuning, class-imbalance weighting, and multiple graph and CNN variants. However, not all of those branches are reported with equally complete comparison tables, so this paper centers its headline claims on the metric files already saved under ‘metrics/’.

Two open pull requests materially affect the near-term CNN story. PR #56 reorganizes CNN training around dedicated student, teacher, training, distillation, and optimization modules and standardizes the 1D input view derived from the shared window tensor. PR #59 layers on multi-teacher distillation, validation-tuned thresholds, and local CPU latency benchmarking, and it reports provisional distilled-student numbers in the pull-request record [5, 6]. Because those changes are not yet merged and their artifacts are not stored under the same ‘metrics/’ convention as the main benchmark, this paper treats them as pending extensions rather than combines them with Table 1.

4.3 Evaluation protocol

The README defines the benchmark as five-class multi-label classification with a nominal decision threshold of 0.5 on the held-out test split. For transparency, the metric implementation also records exact-match accuracy and per-finger summaries, which is important because different models may trade precision for recall while achieving similar binary correctness at the finger level. This default threshold remains the basis for the mainline benchmark tables in this paper; threshold-tuning utilities mentioned from PR #59 are discussed only as extension evidence, not as inputs to the headline comparison.

This paper follows the experimental writing guidance of the external ‘Research-Paper-Writing-Skills’ workflow by tying every headline claim to an existing artifact. Because no ablation package is currently stored in the repository, we deliberately avoid module-level causality claims and instead focus on benchmark-level observations that are directly supported by saved outputs.

5 Results

The current repository benchmark exposes a clear trade-off between balanced sequence modeling and high-recall graph modeling. Table 1 shows that the LSTM achieves the strongest exact-match accuracy, finger accuracy, and macro AUROC, while the GNN narrowly leads on macro F1 and macro AUPRC. The CNN remains competitive as a lightweight baseline but does not surpass the best LSTM or GNN score on any headline metric in the saved artifacts.

The gap between the GNN and the LSTM is easiest to interpret through precision-recall behavior. The GNN posts the highest macro recall among the saved runs (0.832 versus 0.768 for the LSTM and 0.697 for the CNN), which lifts its macro F1 and AUPRC. The LSTM, however, converts that lower recall into materially stronger exact-match behavior and substantially stronger AUROC, suggesting a more balanced discrimination profile across thresholds.

Table 1: Held-out test performance from the repository metric files. Bold numbers mark the best score in each column.

Model	Exact-match	Finger Acc.	Macro F1	Macro AUROC	Macro AUPRC
LSTM	0.545	0.784	0.705	0.858	0.754
GNN	0.448	0.683	0.706	0.787	0.776
CNN	0.442	0.694	0.676	0.767	0.764

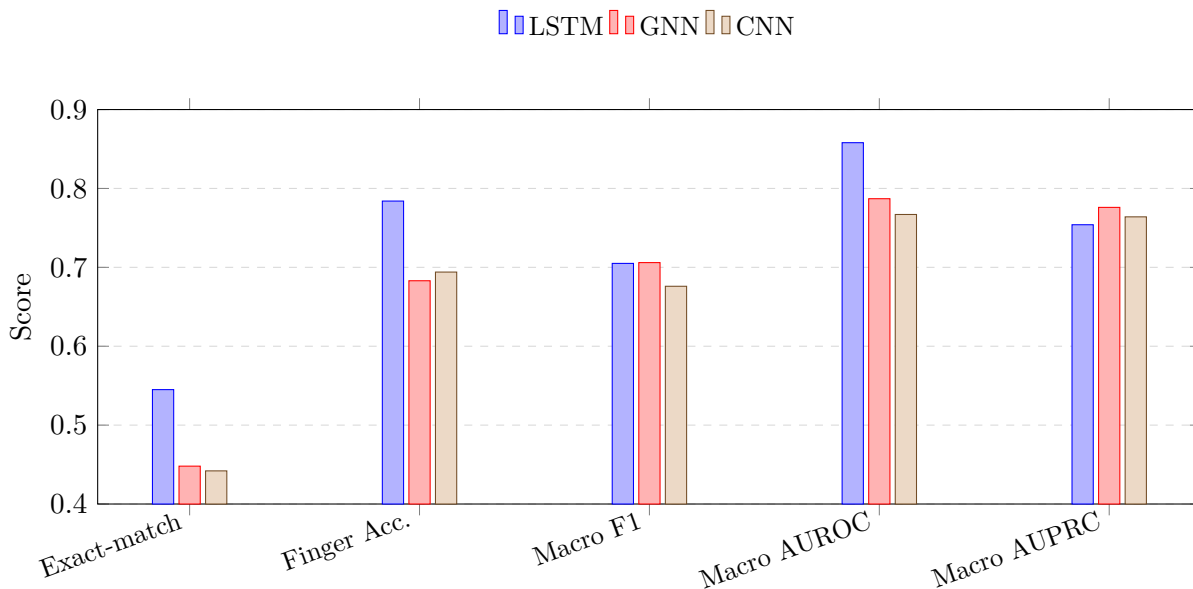


Figure 2: Metric-level comparison across the three saved model families. The GNN leads on recall-sensitive summary metrics, while the LSTM leads on exact-match accuracy, finger accuracy, and macro AUROC.

Open PR evidence suggests that the CNN branch may move closer to the LSTM and GNN frontier once distillation utilities are merged. PR #59 reports a provisional CNN-Micro student distilled from a CNN teacher with macro F1 0.7096, finger accuracy 0.7990, macro AUROC 0.8662, and macro AUPRC 0.7838, improving over the pull request’s stated CNN baseline of 0.6817, 0.7750, 0.8559, and 0.7319 while keeping the student at 51,493 parameters [6]. Because those numbers currently live in an open pull request rather than the committed ‘metrics/’ tree, we report them here as provisional extension evidence instead of merging them into Table 1. The same PR also records local CPU proxy latency of 0.274 ms per sample on average with a 0.505 ms p95, which is useful for screening edge candidates but is not a substitute for hardware-validated rehabilitation timing.

Finger-wise scores show that the ranking is not uniform across all outputs. Table 2 indicates that the GNN is strongest on the thumb label, while the LSTM is strongest on the index, middle, ring, and little finger labels. This pattern is consistent with the repository-wide picture: graph aggregation appears to favor aggressive positive detection, whereas the recurrent model distributes performance more evenly across fingers.

One additional paper-level update is worth noting. The top-level repository README foregrounds the GNN and CNN comparison, but the repository also contains complete LSTM metric artifacts

Table 2: Per-finger F1 score by model family. Bold numbers mark the strongest model for each finger.

Finger	LSTM	GNN	CNN
Thumb	0.716	0.776	0.726
Index	0.723	0.696	0.660
Middle	0.712	0.689	0.675
Ring	0.696	0.688	0.670
Little	0.680	0.678	0.649

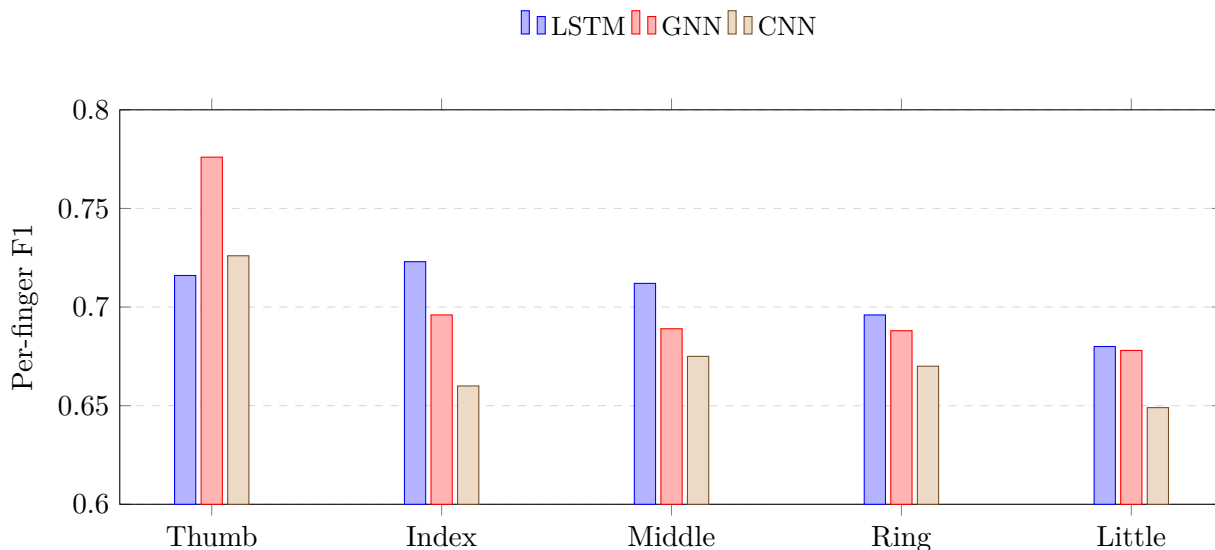


Figure 3: Per-finger F1 comparison derived from the saved metric files. The LSTM wins four of five finger labels, while the GNN is strongest on the thumb class.

under ‘metrics/lstm/'. Including those saved LSTM results materially changes the interpretation of the benchmark, because the best overall system is no longer a simple GNN-versus-CNN choice. At the same time, the open CNN PRs show that this ranking should not be interpreted as final: the repository is still actively evolving toward a lighter-weight distilled branch with stronger deployment-facing evidence.

6 Discussion

The strongest conclusion supported by the present evidence is that PSR-PIPELINE is already valuable as a reproducible benchmarking scaffold even though it is not yet a submission-ready algorithm paper. The preprocessing code, feature representation, and metric artifacts are detailed enough for another engineer or researcher to rerun the benchmark logic, inspect trade-offs, and extend the model suite without reconstructing the full pipeline from scratch.

The benchmark itself reveals two useful modeling lessons. First, a straightforward recurrent model is hard to dismiss in this setting: the saved LSTM run provides the best exact-match decoding quality and the strongest macro AUROC, which are attractive traits for downstream control. Second, the

graph branch is not redundant, because its higher recall and stronger macro AUPRC suggest that explicit cross-window message passing may recover positive finger activations that the other models miss.

The repository also hints at a broader transfer-learning direction. The `run_two_stage.py` script and `results/README.md` file summarize healthy-to-impaired pretraining and finetuning results for CNN and LSTM branches. Those artifacts are promising, especially because rehabilitation deployments often face limited impaired-arm data, but they are packaged differently from the main benchmark tables. For that reason, this paper mentions them as follow-on evidence rather than merging them into the headline comparison.

The open CNN pull requests also clarify the most plausible next paper direction. PR #56 turns the CNN branch into a cleaner systems component with dedicated student, teacher, training, distillation, and optimization modules, while PR #59 adds cross-family teacher ensembling, per-finger threshold calibration, and lightweight latency reporting [5, 6]. If those branches are merged with the same artifact discipline as the current ‘metrics/’ tree, the project could support a stronger edge-deployment narrative than the present mainline benchmark alone.

Several limitations remain. The benchmark currently covers one dataset, one main processed benchmark path, and no stored ablation study for preprocessing or architecture variants. The GNN implementation uses dense window graphs, which may become expensive as sequence length grows. Hardware integration and real-time intent output, which appear in the updated system figure, should therefore be read as downstream integration targets rather than fully validated claims in this paper. The repository also includes a Mamba notebook for signal translation, but that component is not integrated into the same evidence-complete evaluation path as the direct finger-intent predictors. These gaps do not invalidate the current benchmark, but they do define the most important next steps for turning PSR-PIPELINE from a strong engineering base into a stronger empirical paper.

7 Conclusion

This paper documents PSR-PIPELINE as a reproducible workflow for sEMG-based finger-intent prediction in a post-stroke rehabilitation setting. The key idea is not a new standalone network, but a repository structure that makes preprocessing, feature extraction, tensor adaptation, model comparison, and metric reporting explicit and reusable.

The strongest current evidence favors the LSTM for exact-match accuracy, finger accuracy, and macro AUROC, while the GNN offers the best macro F1 and AUPRC. That trade-off makes the repository useful for controlled model comparison rather than single-number leaderboard claims. A current limitation is evaluation scope: the benchmark is still centered on one dataset and lacks stored ablations for several design choices. Extending the same workflow to broader baselines, richer transfer settings, and more clinically realistic validation is the clearest next step.

References

- [1] Formove AI. PhysioMio. <https://huggingface.co/datasets/formove-ai/physiomio>, 2026. Dataset page, accessed July 11, 2026.
- [2] William L. Hamilton, Rex Ying, and Jure Leskovec. Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems*, 2017.

- [3] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.
- [4] Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations*, 2017.
- [5] Post-Stroke Rehabilitation Project. Pull Request #56: New CNN training process. <https://github.com/post-stroke-rehab/psr-pipeline/pull/56>, 2026. Open pull request, accessed July 11, 2026.
- [6] Post-Stroke Rehabilitation Project. Pull Request #59: issue 58 distillation. <https://github.com/post-stroke-rehab/psr-pipeline/pull/59>, 2026. Open pull request, accessed July 11, 2026.
- [7] Post-Stroke Rehabilitation Project. PSR-Pipeline. <https://github.com/post-stroke-rehab/psr-pipeline>, 2026. GitHub repository, accessed July 11, 2026.
- [8] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks. In *International Conference on Learning Representations*, 2018.
- [9] Peter D. Welch. The use of fast fourier transform for the estimation of power spectra: A method based on time averaging over short, modified periodograms. *IEEE Transactions on Audio and Electroacoustics*, 15(2):70–73, 1967.
- [10] Mingde Zheng, Michael S. Crouch, and Michael S. Eggleston. Surface electromyography as a natural human-machine interface: A review. *arXiv preprint arXiv:2101.04658*, 2021.